

Identity-Bound Academic Credentials on Blockchain: On-Chain Issuer Accreditation with ERC-3643 and OnchainID

ANONYMOUS AUTHOR(S)

Verifying academic credentials remains difficult: records are held by individual institutions in proprietary systems, verification is slow and manual, and counterfeit qualifications are widespread. Blockchain-based registries have been proposed as a remedy, but existing systems tend to address only subsets of these requirements: hash-anchoring approaches focus on authenticity, while verifiable identity binding, explicit issuer accreditation, and flexible post-issuance correction workflows are not generally provided together. This paper investigates whether an infrastructure designed for regulated financial instruments can be repurposed to close these gaps. We present the design of a registry for identity-bound academic credentials that composes OnchainID self-sovereign identities (ERC-734/ERC-735) with the T-REX suite (ERC-3643): its trusted-issuer registry becomes an on-chain issuer-accreditation whitelist, and each certificate is represented as a signed, updatable claim bound to a student's identity and verifiable on-chain by any third party without a wallet, with issuer accreditation enforced against registry state at verification time rather than as a client-side policy, while sensitive fields need not be stored on-chain. We make explicit the tension between a transferable-security-token standard and non-transferable credentials, clarifying which of its guarantees carry over. We validate the design with a reference implementation covering the full certificate life cycle and evaluate it in terms of gas cost, scalability, latency, throughput, and security, quantifying the overhead relative to a hash-anchoring baseline. On a four-validator permissioned consortium the design sustains tens of certificate issuances per second with single-block finality.

CCS Concepts: • **Security and privacy** → **Domain-specific security and privacy architectures**; *Digital signatures*; *Authorization*; • **Applied computing** → *Education*.

Additional Key Words and Phrases: academic credentials, self-sovereign identity, issuer accreditation, ERC-3643, OnchainID, verifiable credentials, blockchain

ACM Reference Format:

Anonymous Author(s). 2026. Identity-Bound Academic Credentials on Blockchain: On-Chain Issuer Accreditation with ERC-3643 and OnchainID. *Distrib. Ledger Technol.* 1, 1 (August 2026), 22 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

The management and verification of academic credentials remain persistent problems in the education sector. Records are traditionally held by individual institutions in proprietary systems, so confirming the authenticity of a degree requires contacting the issuing institution through ad hoc, often manual, channels [29]. This opacity has enabled the proliferation of degree mills [11] and counterfeit qualifications [13], which erode trust in legitimate credentials, while the underlying centralized databases remain susceptible to unauthorized alteration and single points of failure.

Blockchain technology has been widely proposed as a remedy [35, 39], and several systems now record credentials on-chain. Broadly, existing approaches fall into three families. *Hash-anchoring* systems, such as Blockcerts [1] and the University of Nicosia's Bitcoin-based diplomas [18], store the cryptographic digest of a certificate on a ledger and verify

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

a document by re-computing and comparing its hash. *Self-sovereign identity* initiatives, most prominently the European Blockchain Services Infrastructure and its identity framework ESSIF [6, 7], issue verifiable credentials governed by a permissioned, institutionally coordinated infrastructure. *Token-based* schemes, such as EduCTX [40], represent credits or credentials as blockchain tokens.

Despite this progress, no existing approach jointly addresses four properties that a practical academic registry requires. First, a certificate should be *cryptographically bound to a verifiable identity* rather than existing as a free-floating document hash. Second, the set of institutions permitted to issue credentials should be governed by an explicit, on-chain *accreditation mechanism*, so that authority to issue is itself verifiable. Third, credentials must be *updatable and revocable*: hash-anchoring systems are issue-only, yet real registries must correct errors and reflect post-issuance changes [12]. Fourth, verification should be possible for a third party *without requiring a wallet or the disclosure of sensitive personal data*, in line with data-protection obligations [43, 46]. Approaches that emphasise authenticity, such as Blockcerts and SmartCert [17], tend to overlook identity binding, updatability, and privacy, while top-down frameworks such as EBSI provide governance but limited flexibility for correction workflows and non-EU adoption [6].

Approach. This paper investigates whether an infrastructure originally designed for *regulated financial instruments* can be repurposed to satisfy these four properties for academic credentials. We build an academic-certificate registry on two composable Ethereum standards: OnchainID [30], a self-sovereign identity framework in which each participant is a smart contract holding signed *claims* (ERC-734/ERC-735 [41, 42]); and the T-REX suite [36], the reference implementation of the ERC-3643 permissioned-token standard for compliant security tokens. The central insight is that the compliance machinery of ERC-3643 maps naturally onto credentialing: its *trusted-issuer registry* becomes an on-chain issuer-accreditation whitelist, and its identity-bound, claim-based model represents each certificate as a signed claim attached to a student’s identity contract. Certificates thus become updatable claims tied to a verifiable identity, issuable only by accredited institutions, and verifiable by any third party against the ledger without a wallet, while sensitive fields are kept off-chain.

Repurposing a security-token standard for credentials also raises a design tension that we make explicit and examine: ERC-3643 targets *transferable* regulated assets, whereas academic certificates are inherently *non-transferable*. We discuss which parts of the standard are exploited (identity, compliance, issuer accreditation) and which are deliberately constrained (transferability), and we compare the resulting overhead against a minimal hash-anchoring baseline to quantify the price of the additional guarantees. We validate the design with a reference implementation, comprising the on-chain contracts and a cross-platform mobile client, and evaluate it in terms of gas cost, scalability, latency, and security.¹

Contributions. This paper makes the following contributions:

- We present a design for an academic-certificate registry that maps the ERC-3643/T-REX compliance model and OnchainID self-sovereign identity onto credentialing, using the trusted-issuer registry as an on-chain issuer-accreditation mechanism and representing certificates as identity-bound, updatable claims (Section 3).
- We articulate and analyse the design tension between a transferable-security-token standard and non-transferable credentials, clarifying which guarantees of ERC-3643 transfer to the credentialing setting and which do not (Sections 3 and 5).

¹The reference implementation is publicly available at <https://anonymous.4open.science/r/certify-app-0F5F/>.

- We describe a reference implementation of the complete certificate life cycle (institution accreditation, identity creation, issuance, update, and wallet-free validation) including a privacy-preserving scheme that keeps sensitive fields off-chain (Section 4).
- We provide an empirical evaluation covering per-operation gas cost and its monetary cost across deployment scenarios, scalability with respect to registry size and claims per identity, and, measured on a four-validator Hyperledger Besu QBFT (Quorum Byzantine Fault Tolerant) consortium, sustained certificate-issuance throughput and confirmation latency, together with a threat-model-based security analysis and a quantitative comparison with a hash-anchoring baseline (Section 5).

The remainder of the paper is organised as follows. Section 2 reviews the necessary background on blockchain and credentialing and positions our work with respect to existing approaches. Section 3 presents the design of the registry and its mapping onto OnchainID and T-REX. Section 4 describes the reference implementation. Section 5 reports the evaluation, and Section 6 concludes.

2 Background and Related Work

This section provides the background needed to follow the proposed design and positions it with respect to existing work. We keep the treatment of general blockchain concepts brief and concentrate on the on-chain identity and token standards on which our approach builds, and on the landscape of blockchain-based academic credentialing.

2.1 Blockchain and Smart Contracts

A blockchain is a distributed, append-only ledger maintained by a network of peers without a central authority. Records are grouped into blocks, each cryptographically linked to its predecessor through a hash, so that any modification of a past record invalidates every subsequent block; agreement on the ledger state is reached through a consensus mechanism [27, 48]. These properties make the ledger tamper-evident and independently verifiable, which is precisely the guarantee sought for academic records that today rely on a single institution [29]. Public, permissioned, and consortium deployments trade off openness against control [33]; our work targets an Ethereum-compatible setting [2, 47], which can be instantiated on the public network, a Layer-2, or a private consortium chain.

Smart contracts are programs executed deterministically by every node on the Ethereum Virtual Machine. They allow issuance and verification logic to run without a trusted intermediary, but their state, once written, is effectively immutable, a property that underpins integrity yet conflicts with data-protection requirements such as the right to erasure, a tension we address through off-chain storage of sensitive data (Section 4).

2.2 On-Chain Identity and Token Standards

Our design composes two Ethereum standards. *OnchainID* [30, 31] is a self-sovereign identity framework in which every participant, an individual or an organisation, is represented by an identity smart contract [26]. These contracts implement ERC-734 (key manager) [42] and ERC-735 (claim holder) [41], which respectively define how cryptographic keys and signed *claims* are managed. A claim is an assertion about an identity, for example a name or a qualification, signed by an issuer and attached to the subject's identity contract; it can be self-attested or delegated to an authorised issuer.

T-REX [36] is the reference implementation of ERC-3643, a standard for *permissioned tokens* that represent regulated assets. Beyond the token itself, the suite provides a compliance layer whose central component, for our purposes, is a

trusted-issuer registry: an on-chain whitelist recording which identities are authorised to issue claims for which topics. Although ERC-3643 was conceived for transferable security tokens, its identity-bound, claim-based compliance model is what we repurpose for credentialing; the mismatch with the *non-transferable* nature of certificates, which are closer to *soulbound* tokens [45] and to non-transferability standards such as ERC-5192 [4], is examined in Sections 3 and 5.

2.3 Blockchain-based Academic Credentialing

Existing systems that place academic credentials on-chain fall into three families.

Hash-anchoring. The most common approach stores only the cryptographic digest of a certificate on the ledger and verifies a document by re-computing and comparing its hash. Blockcerts [1] and the University of Nicosia's Bitcoin-based diplomas [18, 35] follow this model, as does SmartCert [17], which adds cryptographic signatures for use during recruitment. These systems establish authenticity effectively but are issue-only: they provide no mechanism for correcting or revoking a certificate, do not bind credentials to a verifiable on-chain identity, and typically require the holder to share a certificate hash with the verifier, which is neither convenient nor privacy-preserving.

Self-sovereign identity and verifiable credentials. A growing body of work applies self-sovereign identity and W3C verifiable credentials [37, 38], whose data model reached version 2.0 in 2025, to education [3]. At the infrastructural level, the European Blockchain Services Infrastructure (EBSI) and its identity framework ESSIF [6, 7] issue verifiable credentials over a permissioned, institutionally governed infrastructure, providing strong interoperability and governance across member states; this direction is reinforced by the revised eIDAS regulation (eIDAS 2.0) [8], which mandates interoperable European Digital Identity (EUDI) Wallets for holding and presenting credentials such as diplomas. However, this top-down, EU-centric model offers limited flexibility for post-issuance correction workflows and for adoption outside its governance perimeter. The QualiChain case study [12] integrates an academic management system (FenixEdu) with Ethereum to register diploma hashes; it demonstrates real-world feasibility and user acceptance but again centres on authenticity rather than identity binding, updatability, or privacy.

Token-based. EduCTX [40] represents study credits as blockchain tokens under a unified ECTS-based scheme, standardising credit management. It does not, however, address certificate issuance, identity management, or selective disclosure, and verification requires the holder to supply a multi-signature address and redeem script. Other token- and smart-contract-based registries have been proposed [34] with similar scope.

2.4 Positioning

Table 1 contrasts representative systems on the four properties identified in Section 1. No prior approach jointly provides identity binding, an on-chain issuer-accreditation mechanism, updatable and revocable credentials, and wallet-free verification with off-chain protection of sensitive data. Our approach attains all four by combining OnchainID identities with the ERC-3643 trusted-issuer registry, at a cost we quantify against a hash-anchoring baseline in Section 5.

2.5 Adoption Challenges

Beyond technical design, deploying blockchain in education faces well-documented barriers. Public institutions operate under legal and administrative constraints and legacy systems that make procedural change costly, and adoption is further slowed by administrative resistance and the need for specialised expertise and training [19, 21, 32]. Data protection is a central concern: regulations such as the GDPR grant a right to erasure that conflicts with the immutability

Table 1. Comparison of representative blockchain-based credentialing approaches. “Yes”/“No” denote support or its absence; “–” denotes partial support or a property out of the approach’s scope. Blockcerts supports revocation via an issuer-hosted list [1], but not in-place correction.

Approach	Identity binding	On-chain issuer accreditation	Updatable / revocable	Wallet-free verification	Off-chain private data
Blockcerts / UNIC [1, 18]	No	No	–	–	–
SmartCert [17]	No	No	No	No	No
EBSI / ESSIF [6, 7]	Yes	–	–	–	Yes
QualiChain [12]	No	–	No	–	–
EduCTX [40]	–	No	No	No	No
Our approach	Yes	Yes	Yes	Yes	Yes

of on-chain records [43, 46], motivating hybrid designs that keep personal data off-chain and store only integrity-preserving references on-chain. Scalability and cost are likewise practical considerations, as credential verification volumes can be high [22, 49]; these motivate the deployment scenarios and the empirical cost analysis reported in Section 5.

3 Registry Design

This section presents the design of the proposed academic-certificate registry. We first outline the overall architecture, then describe the central contribution, the mapping of the OnchainID and T-REX (ERC-3643) primitives onto credentialing, and the design decisions that follow from it: how a transferable-security-token standard is adapted to non-transferable credentials, how personal data is protected, and how the certificate life cycle is realised. Implementation details are deferred to Section 4.

3.1 Architecture Overview

The system is organised in two layers (Figure 1). The *on-chain layer* holds the registry logic and state as smart contracts on an Ethereum-compatible network; it is where identity, accreditation, and certificate claims live, and it is the focus of this paper. The *client layer* is a cross-platform application through which students, institutions, and verifiers interact with the on-chain layer; its implementation is described in Section 4. Confining all trust-bearing logic to the on-chain layer means that verification does not depend on the client: a third party can check a certificate directly against the ledger.

3.2 Mapping Credentialing onto OnchainID and T-REX

The core of the design is the observation that the identity and compliance primitives of OnchainID and ERC-3643, though conceived for regulated financial assets, correspond closely to the elements of an academic registry. Table 2 summarises the correspondence.

Each participant is represented by an OnchainID identity contract, so that both students and issuing institutions have a unique, verifiable on-chain identity [30, 31]. A certificate is modelled not as a standalone document but as a *claim* (ERC-735 [41]), a statement signed by an issuer and attached to the subject’s identity contract. Composite certificates are expressed as several claims (e.g. name, student number, course, completion date, registration number, and the certificate digest), each carrying the issuer’s signature.

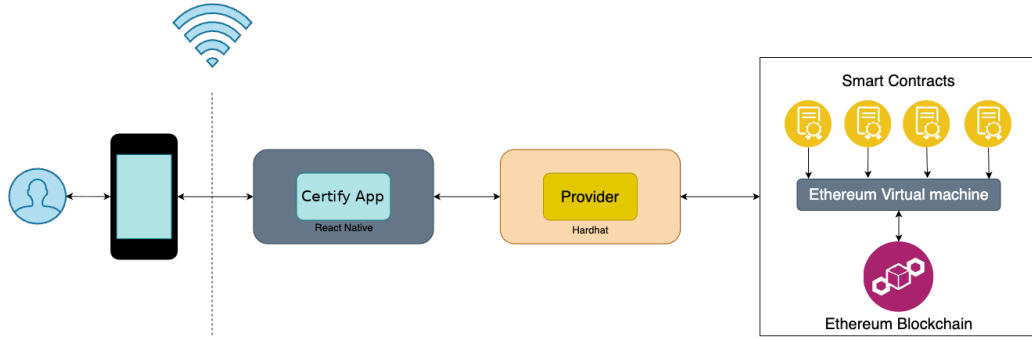


Fig. 1. Architecture overview.

Table 2. Mapping of OnchainID/ERC-3643 primitives onto the academic registry.

Primitive	Role in the registry
OnchainID identity contract (ERC-734/735)	Verifiable on-chain identity of a student or an institution
Signed claim (ERC-735)	A certificate, or a field of it, bound to the student's identity
Claim topic	Category of asserted data (institution, student, certificate)
Trusted-issuer registry (ERC-3643)	On-chain accreditation whitelist of institutions authorised to issue
Claim / issuer key (ERC-734)	Authorisation for an institution to write claims on an identity

Authority to issue is made verifiable through the ERC-3643 trusted-issuer registry [36], an on-chain issuer-accreditation whitelist recording which institutions, and for which claim topics, are accredited. Following the ERC-735 claim model, writing a claim to a student's identity is gated on-chain by an ERC-734 claim key authorised on that identity (ERC-734 [42]); the trusted-issuer registry is then decisive *at verification time*, where a claim counts as a valid certificate only if it is signed by an issuer that is *currently* accredited in the registry for its topic. Accreditation is thus an on-chain invariant of verification rather than a client-side policy: a claim written by a party that is not accredited (even one that holds a claim key and calls the identity contract directly, bypassing the client) simply fails verification, and a claim from an issuer that is later de-accredited stops verifying from that point on. This is the essential difference from hash-anchoring registries, in which an anchored digest carries no on-chain notion of who was authorised to record it. We make this verification concrete as a single read-only on-chain call in Section 3.5, and evaluate it against an explicit threat model in Section 5.6. To manage the many identity and issuer contracts uniformly, the design follows the factory and proxy patterns [14, 15] recommended by the frameworks, the latter allowing the certificate-management logic to be upgraded without disturbing existing data.

3.3 Adapting a Security-Token Standard

ERC-3643 targets *transferable* regulated securities, whereas academic certificates are inherently *non-transferable*, resembling *soulbound* tokens [45]: a credential is meaningful only when bound to the individual who earned it. The

design therefore exploits the parts of the standard that concern *identity and compliance* (verified on-chain identities, the trusted-issuer registry, and the claim-topic model) while deliberately not using the token’s transfer semantics as a means of moving credentials between holders. In effect, the compliance infrastructure of a security-token standard is reused as an accreditation-and-identity layer, and the transferability it was designed to regulate is constrained away. What the approach gains from ERC-3643 is thus the ready-made, audited machinery for identity binding and issuer whitelisting; what it does not inherit is a use for free transfer. We return to this trade-off quantitatively in Section 5, where the cost of this machinery is compared against a bare hash-anchoring baseline.

3.4 Privacy by Design

Because on-chain state is effectively immutable and publicly readable, personal data cannot be written to it in the clear without conflicting with data-protection obligations such as the right to erasure. The design therefore distinguishes *public* fields (those needed for direct verification, which are stored on-chain and are, by design, visible to anyone) from *sensitive* fields, which are kept off-chain and encrypted (AES-256) under a symmetric key shared between the student and the issuing institution; for the latter, only an integrity-preserving reference (a hash) is recorded on-chain, so that the ledger attests to their integrity without holding their contents. This is a deliberate scoping rather than blanket confidentiality: the scheme protects the sensitive fields, not the public ones it deliberately exposes for verification. Two limits follow and we state them plainly. First, the on-chain reference is a hash of possibly low-entropy data and should be salted to resist dictionary attack, a hardening we recommend and assess in Section 5.6. Second, disclosure is selective only at the granularity of individual claims: to a verifier can be shown one certificate claim without the others on the same identity, but not an arbitrary sub-field within a single claim. Verifying a public or referenced field requires neither a wallet nor the disclosure of the sensitive off-chain data. The concrete storage and hashing scheme is described in Section 4.

3.5 Certificate Life Cycle

The registry supports three protocols: issuance, update, and validation.

Issuance. A student requests a certificate from an accredited institution (Figure 2). After the student’s identity is created and the institution’s issuing key is authorised on it, the institution, having been resolved from the trusted-issuer registry for the relevant claim topic, signs and writes the certificate claims to the student’s identity. The certificate is thereafter a verifiable, identity-bound record on the ledger.

Update and revocation. Unlike issue-only hash-anchoring systems, the registry supports post-issuance correction (Figure 3). A student initiates a change request; upon approval, the accredited institution issues updated claims, superseding the previous ones. Because claims are managed by authorised keys, corrections and revocations remain gated by the same accreditation checks as issuance, and every change is recorded on the ledger.

Validation. A third party verifies a certificate by presenting it to an on-chain check that returns whether the student’s identity holds a matching, authentically issued, and accredited certificate claim (Figure 4). The verifier first computes the certificate’s digest, a public operation that requires no trust, and then invokes a read-only on-chain view that, consulting only ledger state, confirms three things at once: that a claim on the student’s identity binds the presented certificate, that the claim’s signature is valid, and that its issuer is *currently* accredited in the trusted-issuer registry for the certificate topic. Because these trust-bearing checks (accreditation and signature) are computed on-chain from

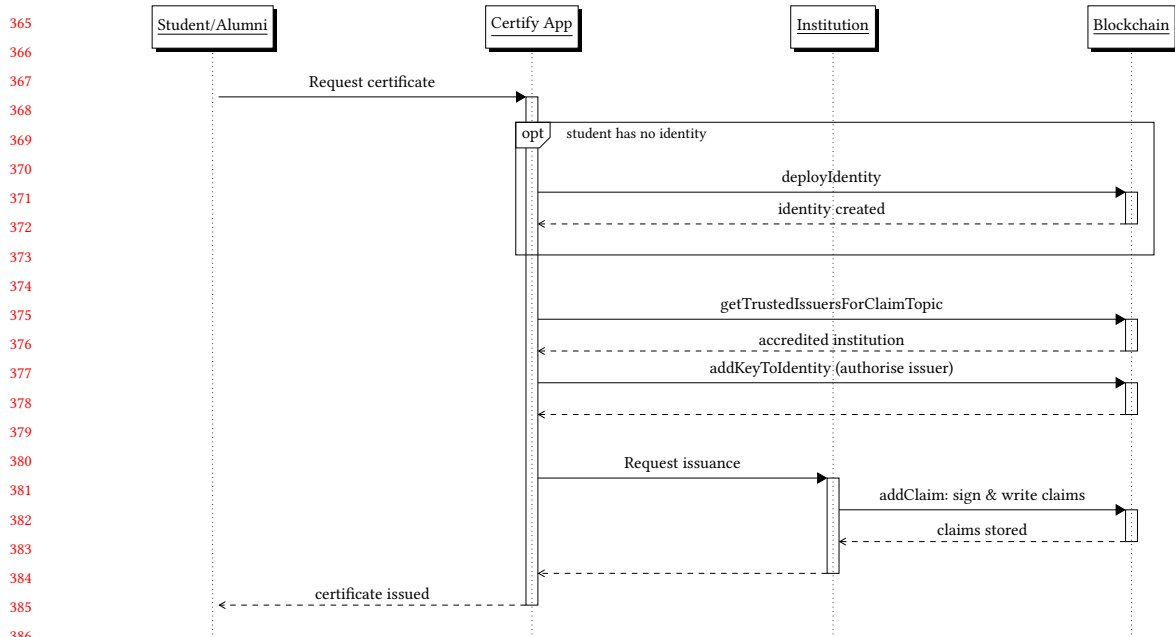


Fig. 2. Issuance of an academic certificate.

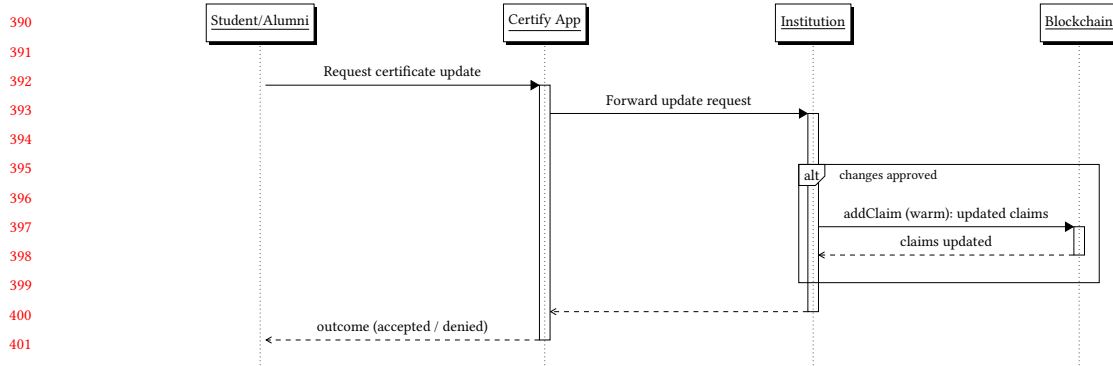


Fig. 3. Requesting an update to an academic certificate.

registry and identity state rather than in the client, the verdict is authoritative and cannot be bypassed by a client that skips them: it requires no wallet, exposes no private data, and does not depend on the issuing institution being reachable. A claim whose issuer was never accredited, or has since been de-accredited, fails this check even though it remains stored on the identity.

3.6 Security Considerations

The registry's guarantees rest on three mechanisms established above: the tamper-evidence and immutability of the underlying ledger and its EVM-based execution [47]; the on-chain gating of claim writes by per-identity key

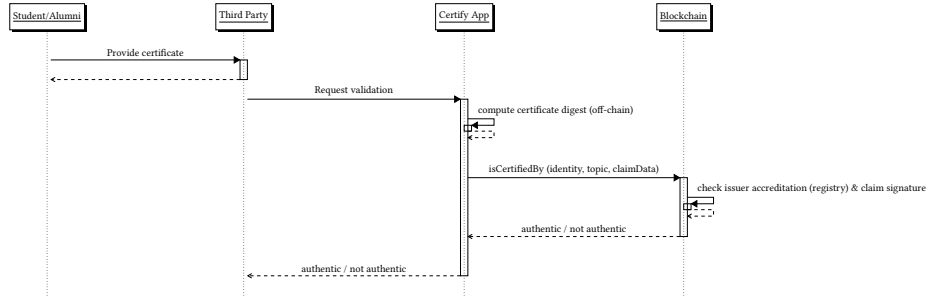


Fig. 4. Validation of an academic certificate. The trust-bearing checks, issuer accreditation and claim signature, are performed on-chain by the read-only `isCertifiedBy` view; the digest computation and claim matching are performed off-chain, so the verdict does not depend on the client.

authorisation, combined with the enforcement of issuer accreditation and signature validity *at verification time*, so that a claim counts as a valid certificate only if it is issued by a currently accredited issuer: a claim from an unaccredited party, however it was written, does not verify; and the signing of every claim by its issuer, which makes forgery detectable. Residual risks (such as issuer-key compromise, denial of service through registry growth, and privacy leakage via on-chain references) are analysed under an explicit threat model, together with the corresponding mitigations, in Section 5.

4 Reference Implementation

We validate the design with a reference implementation comprising the on-chain contracts and a cross-platform client that realises the complete certificate life cycle. This section describes how the design of Section 3 is realised in code; the implementation is released as open source.² We focus on the elements that embody the design (the claim-based data model, the accreditation and issuance paths, and the privacy-preserving storage and validation scheme) rather than on the user interface.

4.1 Overview

The on-chain layer reuses two contract suites. OnchainID identities are deployed and managed through the `@onchain-id/identity-sdk` package, using its `IdentityFactory` to instantiate identity contracts uniformly. The T-REX (ERC-3643) suite, for which no equivalent package is provided, is deployed from imported contracts by a dedicated script. All contracts are compiled and deployed with Hardhat [28] (Solidity 0.8.17/0.8.20, optimizer enabled, runs = 200); the deployment emits each contract's address and ABI to a configuration file consumed by the client. The client targets a single network through the RPC endpoint recorded in this configuration file; retargeting it to another chain (e.g. a consortium or Layer-2 network) amounts to regenerating the configuration against the new endpoint and redeploying the contract suite there, rather than switching networks at run time. The client is a React Native [5, 20]/Expo [9] application in JavaScript [25] that interacts with the chain through ethers.js [23, 24]; wallet keys are held in the device secure enclave via Expo Secure Store [10], and wallet connections use WalletConnect [44]. Contract deployment and the generation of the configuration file are automated by an installation script accessible only to the registry owner, so that the creation of accredited issuers remains a privileged operation.

²<https://anonymous.4open.science/r/certify-app-0F5F/>

4.2 Claim-based Certificate Model

Certificates are represented as ERC-735 claims under three topics: INSTITUTION, STUDENT, and CERTIFICATE, each carrying a subset of the certificate fields (for example, the institution’s official code and course identifier, the student’s name and number, and the certificate reference). A claim records its topic (the keccak256 digest of the topic name), a signature scheme (ECDSA), the issuer identity, the issuer’s signature, the claim data, and a `uri` field; its identifier is derived as `keccak256(issuer, topic)`. Each claim is signed by the issuing institution over the tuple `(identity, topic, data)`, so that any party can later verify both the content and its origin. This realises directly the identity-bound, signed-claim model of Section 3.2.

4.3 Issuer Accreditation and Identity Creation

Institutions are onboarded as *claim issuers*. The `deployClaimIssuer` routine deploys a claim-issuer contract for an institution’s wallet and records it in the trusted-issuer registry through `addTrustedIssuer`, together with the set of claim topics it is permitted to issue; the institution then registers its official code for later lookup. This is the on-chain issuer-accreditation step, and, being performed by the installation script, is restricted to the registry owner.

A student’s identity is created by `deployIdentity`, which deploys an identity contract for the student’s wallet using a unique salt and confirms creation by observing the corresponding on-chain event. Before an institution may write claims to a student’s identity, its key is authorised on that identity via `addKeyToIdentity`, implementing the second, per-identity level of the two-level gating described in Section 3.2.

4.4 Issuance

Claims are added through `addClaim`. At the application layer this first verifies that the issuer is present in the trusted-issuer registry (or is the identity’s self-signer), rejecting the request otherwise; on-chain, the identity contract independently requires that the sender hold an authorised claim key (`onlyClaimKey`) and that the issuer’s signature be valid (`isClaimValid`). Accreditation is thus checked at the client while key authorisation and signature validity are enforced on-chain, realising the two-level gating of Section 3.2. To request a certificate, the client resolves the accredited issuer for the relevant topic (`getTrustedIssuersForClaimTopic`) and records self-attested student claims (name and number). Issuance by an institution proceeds identically, with the institution signing and adding the remaining claims: institution and course data, the certificate reference, and the certificate itself. Updating a certificate reissues the affected claims under the same checks, so corrections remain gated by accreditation.

4.5 Privacy-Preserving Storage

The implementation separates public from sensitive data, as required by the design of Section 3.4. Public fields are stored on-chain in the clear to permit direct verification. Sensitive fields are encrypted with AES-256 under a symmetric key shared between the student and the issuing institution before being placed in the claim, so that only the intended parties can read them. The certificate document itself is kept off-chain; what is anchored on-chain is its SHA-256 digest, stored in the claim’s `uri` field. The ledger therefore attests to a certificate’s integrity without exposing its contents, keeping personal data off-chain while preserving verifiability.

4.6 Validation

Validation requires neither a wallet nor the issuing institution to be online. A verifier presents the certificate (as a link or its contents); the client retrieves the certificate claims of the student's identity (`getClaimsByTopic`) and recomputes the SHA-256 digest of the presented certificate, using it to identify the corresponding stored claim. The client then invokes the read-only `CertificateVerifier.isCertifiedBy` view with the matched claim data, which checks the claim data, the claim signature, and the issuer's current accreditation. Because this is a read-only check against on-chain state, any third party can independently confirm authenticity without holding a wallet or accessing the private claim data.

5 Evaluation

This section evaluates the proposed academic-certificate registry built on OnchainID and the T-REX (ERC-3643) suite, described in Sections 3 and 4. Rather than assessing the mobile client, we focus on the on-chain layer, which determines the feasibility, cost, scalability, and security of the approach. The evaluation is organised around five research questions:

- RQ1 Feasibility.** What is the gas cost of each operation, and the resulting monetary cost under realistic deployment scenarios?
- RQ2 Overhead.** How much more expensive is the identity- and compliance-based design than a bare hash-anchoring baseline, and what does the additional cost provide?
- RQ3 Scalability.** How do issuance and validation costs grow as the trusted-issuer registry and the number of claims per identity increase?
- RQ4 Performance.** What end-to-end latency and throughput can the system sustain?
- RQ5 Security.** Against a defined threat model, which attacks are mitigated, and what residual risk remains?

5.1 Experimental Setup

We evaluate the system on three environments, each targeting a different class of metric. Gas consumption is deterministic for a given bytecode, input, and state, and is therefore measured once on a local development network; monetary cost is then modelled across chains and gas prices, public-network latency is measured on a live testnet, and sustained throughput and confirmation latency are measured on a permissioned consortium.

- **Local development network.** A Hardhat [28] node, used to obtain deterministic gas measurements (RQ1–RQ2) and to run the scalability sweeps (RQ3).
- **Public testnet.** The Ethereum Sepolia testnet, used to measure end-to-end latency and to confirm that gas usage matches the local network (RQ4).
- **Permissioned consortium.** A four-validator Hyperledger Besu [16] network running the QBFT consensus, a Byzantine-fault-tolerant protocol tolerating f faulty validators out of $3f+1$ (here $f=1$), used to profile sustained certificate-issuance throughput and confirmation latency on a realistic inter-institutional deployment (RQ4). The validators run as Docker containers on 2-vCPU/4 GB Ubuntu 24.04 virtual machines, split 2+2 across two OpenStack regions of a university private cloud (cross-region round-trip $\approx 2\text{--}5$ ms). The chain uses a 30 M-gas block limit and a zero gas price; the QBFT block interval is a deployment parameter that we sweep (1, 2, and 5 s).

This permissioned consortium, rather than a public chain, is the realistic inter-institutional target deployment, and we profile its sustained throughput and latency directly in Section 5.5.

Table 3. Deployment cost and runtime bytecode size of the implementation contracts, reported against the EIP-170 limit (24,576 bytes). These are one-time costs, amortised across all users; in the live suite they are deployed behind proxies, and per-student identities are proxies created through the factory at 456,619 gas (Table 4). The total excludes the proxies, the implementation authority, and the factory.

Contract	Deployment gas	Bytecode (bytes)	% of EIP-170
Token (ERC-3643)	3,133,123	14,245	58.0
IdentityRegistry	1,545,748	6,910	28.1
IdentityRegistryStorage	1,033,016	4,534	18.4
TrustedIssuersRegistry	1,176,500	5,201	21.2
ClaimTopicsRegistry	482,260	1,987	8.1
ModularCompliance	1,266,889	5,619	22.9
Identity (implementation)	3,837,535	17,383	70.7
ClaimIssuer (implementation)	4,533,082	19,987	81.3
Implementation total (one-time)	17,008,153	–	–

All measurements were obtained with Solidity 0.8.17/0.8.20, optimizer enabled with runs = 200, and each compiler’s default EVM target (no explicit evmVersion was set); the exact toolchain versions and the commit hash of the evaluated code are reported in the public repository.³ Each recurring state-changing operation was executed over $R = 30$ independent runs using fresh inputs (a new wallet and salt per run); we report the median together with the minimum and maximum, and deterministic one-time deployments were sampled over three runs. Where an operation writes to previously unset storage, we report the first-write (*cold*) and rewrite (*warm*) cost separately, as the two differ substantially at the EVM level.

5.2 Cost Analysis (RQ1)

We first measure the one-time deployment cost of the contracts that constitute the registry. Table 3 reports the gas consumed and the deployed bytecode size of each contract, the latter compared against the EIP-170 limit of 24,576 bytes. The two OnchainID identity contracts dominate: the ClaimIssuer and Identity implementations reach 81% and 71% of the code-size limit, respectively, whereas the T-REX registries and the token remain below 60%. Their size is the reason both are deployed behind proxies, and it foreshadows the onboarding asymmetry discussed below: an institution’s ClaimIssuer is deployed as a full contract, whereas a per-student identity is instantiated as a lightweight proxy through the factory.

Table 4 reports the gas cost of the recurring operations that make up the certificate life cycle. The two composite rows, issuing and updating a certificate, aggregate the underlying calls and represent the numbers of practical interest. Certificate validation uses an off-chain digest comparison to identify the corresponding claim, followed by a read-only on-chain view (isCertifiedBy) that composes, in a single call, the claim-data check, the claim-signature check, and the issuer-accreditation check against the current trusted-issuer registry (Section 3.5). Because it is a view, an external caller pays no gas: verification runs as an eth_call, off the transaction path. The 110,939 gas we measured is therefore not a cost borne by verifiers but the view’s on-chain execution cost, the gas it would consume only if the same call were made from within a transaction, e.g. by another contract. Its client-observed latency is analysed in Section 5.4. Adding a claim to a previously unset (issuer, topic) slot (*cold*) costs 355,409 gas, whereas rewriting an existing claim (*warm*),

³The measurement harness and raw data are available at <https://anonymous.4open.science/r/certify-app-0F5F/>.

Table 4. Gas cost of recurring certificate life-cycle operations (medians on the local network over 30 runs). Cold and warm denote first-write and rewrite storage costs, respectively. The composite rows are sums of their components: issuance authorises the issuer once (addKeyToIdentity) and writes the certificate claim (addClaim cold), while an update rewrites that claim (addClaim warm). The identity-proxy onboarding row is the lightweight alternative to deployClaimIssuer (createIdentity+claim-key+addTrustedIssuer), discussed in the text. Validation is a read-only view and costs no gas for an external caller; its on-chain execution cost, incurred only if invoked from another contract, is 110,939 gas for a single accredited issuer.

Operation	Gas (cold)	Gas (warm)
deployIdentity (student)	456,619	n/a
deployClaimIssuer (institution)	4,881,061	n/a
Onboard institution (identity proxy)	815,083	n/a
addTrustedIssuer	240,611	n/a
addKeyToIdentity	163,518	n/a
addClaim (single claim)	355,409	122,105
Issue certificate (composite)	518,927	n/a
Update certificate (composite)	n/a	122,105
Validate certificate (on-chain isCertifiedBy view)	0 (read-only)	

which is also the certificate-update path, costs only 122,105 gas: a 2.9× reduction that reflects the difference between initialising and updating storage at the EVM level.

Two asymmetries stand out. First, onboarding an institution is an order of magnitude more expensive than onboarding a student: deployClaimIssuer consumes 4,881,061 gas (10.7× the 456,619 of deployIdentity) because each claim issuer is deployed as a full contract, whereas student identities are instantiated cheaply through the factory and proxy pattern. Onboarding the institution as a lightweight identity proxy instead removes most of this cost (815,083 gas; Table 4), as we detail below. Second, issuing a certificate for the first time from a given institution (authorising its key, then writing the certificate claim) costs 518,927 gas, but every subsequent certificate from the same institution costs only a single claim (355,409 gas, or 122,105 for an update), since the authorisation persists on the student’s identity.

These numbers raise a natural question: why inherit the footprint of a security-token standard (the ClaimIssuer and Identity implementations alone reach 81% and 71% of the EIP-170 limit) rather than implement a minimal contract that provides identity binding and issuer accreditation directly? The choice is deliberate reuse rather than economy of bytecode. OnchainID and T-REX are widely deployed, externally audited implementations of the ERC-734/735 and ERC-3643 standards; building on them yields a claim-and-accreditation model that is interoperable with the broader ERC-3643 ecosystem and inherits its audited access-control logic, at the price of carrying token machinery the credential setting never exercises. A bespoke contract would shed that machinery and lower deployment cost, but at the cost of a new, unaudited trust base and no standard interoperability, a trade-off we make explicit here and revisit in Section 6.

The main run-time cost this bloat imposes is the one-time, per-institution deployClaimIssuer (4.9M gas), which deploys a full ClaimIssuer contract for each institution. A lightweight alternative is to onboard an institution exactly as a student is, as a factory-created identity proxy, and simply record that identity in the trusted-issuer registry: because isClaimValid (signature recovery plus the claim-key purpose check) is already provided by the base Identity contract, such a proxy can sign and be accredited without deploying any per-institution bytecode. Measured directly in the harness, this alternative (a createIdentity, a claim-key authorisation on the issuer’s identity, and an addTrustedIssuer) onboards an institution for a median of 815,083 gas, a 6.0× reduction from the 4,881,061 of deployClaimIssuer, at the price of forgoing the ClaimIssuer-specific *per-claim signature-based revocation* that the on-chain verifier

Table 5. Modelled monetary cost (USD) of the measured operations across deployment scenarios, from the gas of Table 4 at an assumed Ether price of \$3,000. Mainnet columns assume the indicated gas price; the Layer-2 column assumes 0.1 gwei; the private chain incurs only operational cost.

Operation	Mainnet 10 gwei	Mainnet 30 gwei	Mainnet 100 gwei	Layer-2 0.1 gwei	Private chain
Onboard institution (deployClaimIssuer)	146.43	439.30	1,464.32	1.46	0
Onboard institution (identity proxy)	24.45	73.36	244.52	0.24	0
Onboard student (deployIdentity)	13.70	41.10	136.99	0.14	0
Issue certificate (composite)	15.57	46.70	155.68	0.16	0
Update certificate (warm)	3.66	10.99	36.63	0.04	0
Validate certificate (off-chain)	0	0	0	0	0

honours for full-ClaimIssuer issuers (Section 5.6). For the removal-based revocation the system uses by default (removeClaim and removeTrustedIssuer), both onboarding paths behave identically; they differ only in that the full ClaimIssuer additionally supports per-claim signature revocation, which isCertifiedBy then enforces at verification. The implementation supports both paths, so the lightweight one is available wherever its lower cost outweighs that capability. Independently, for budget-constrained public institutions a consortium or Layer-2 deployment, where this gas carries little or no monetary cost (Table 5), already removes the practical friction.

Because gas price and the fiat exchange rate are volatile, we report monetary cost as a function of the deployment scenario rather than as a single value. Table 5 converts the measured gas of the core operations into fiat cost across a public-chain gas-price range (10–100 gwei), a representative Ether-denominated Layer-2 rollout (0.1 gwei), and a private consortium chain on which gas carries no monetary cost. We assume an Ether price of \$3,000; the gas price and this rate are illustrative, and the ordering across scenarios is rate-independent. On the public chain, the recurring operations are inexpensive (issuing a certificate costs roughly \$16–156 and onboarding a student \$14–137), while the one-time onboarding of an institution is the only costly step (\$146–1,464), for the structural reason noted above, though the lightweight identity-proxy alternative cuts this to \$24–245. On a Layer-2 every operation costs at most a few cents, and on a private consortium chain gas carries no monetary cost. A mainnet deployment is therefore an upper bound on cost, and a consortium or Layer-2 deployment is the economically realistic target.

5.3 Comparison with a Hash-Anchoring Baseline (RQ2)

To quantify the overhead of the identity- and compliance-based design, we compare it against a minimal hash-anchoring baseline in the style of Blockcerts [1], in which the SHA-256 digest of a certificate is recorded in a registry contract and verified by lookup. Table 6 contrasts the two designs on cost and on the functional properties they provide. The comparison isolates the price paid for identity binding, issuer accreditation, updatable credentials, and wallet-free verification.

The baseline reduces issuance to a single storage write of a digest, and is correspondingly cheap (45,542 gas per certificate) but it anchors an opaque hash that is bound neither to the subject who earned the credential nor to an accredited issuer: any account may record any digest, and there is no gated correction or revocation. Our approach issues an identity-bound, issuer-gated certificate for 518,927 gas (the composite of authorising the issuer’s key and writing the certificate claim, excluding the amortised one-time onboarding of the identity and the issuer). Both designs validate at no gas cost to the caller and without a wallet, in our approach through the read-only isCertifiedBy view. The identity and compliance layer therefore costs roughly an order of magnitude in issuance gas (about 11× the baseline). What this

Table 6. Our approach versus a hash-anchoring baseline. Gas values are per operation; feature rows indicate whether the property is supported.

Metric / Property	Hash anchoring	Our approach
Gas to issue a certificate	45,542	518,927
Gas to validate	0	0
Identity binding	No	Yes
Issuer accreditation	No	Yes
Updatable / revocable	No	Yes
Wallet-free verification	Yes	Yes
Selective disclosure of fields	No	Partial

buys is not incidental metadata but the property the baseline structurally cannot provide: a verifier can obtain, from on-chain state alone, an authoritative answer to “was this certificate issued by an institution that is *currently* accredited for this credential type, bound to the named subject, and not since revoked?”, exactly the question that matters for a credential and exactly the one a bare digest anchor leaves unanswerable, since it records only that *some* account once wrote *some* hash. The overhead is thus the price of turning “a hash exists on-chain” into “an accredited issuer certified this subject,” a one-time-per-certificate cost that is constant and, at typical layer-2 or consortium gas prices, negligible in absolute terms.

5.4 Scalability (RQ3)

We assessed how cost evolves along two axes: the size of the trusted-issuer registry and the number of claims held by a single identity. Measurements were taken on the local network, where gas is deterministic; each configuration was sampled over $R = 10$ repetitions, and we report medians.

Trusted-issuer registry. Resolving the set of accredited issuers for a claim topic (`getTrustedIssuersForClaimTopic`) is on the verification path, so we grew the registry from 5 to 45 issuers and measured the execution gas of this resolution (Figure 5). The cost is linear in the number of registered institutions, rising from 56,337 gas at 5 issuers to 147,053 at 45: a slope of approximately 2,268 gas per issuer, i.e. resolution gas $\approx 4.5 \times 10^4 + 2.27 \times 10^3 n$. This linear term is, however, bounded: the T-REX `TrustedIssuersRegistry` rejects any addition beyond 50 issuers (`require(_trustedIssuers.length < 50)`), so resolution never exceeds about 150,000 gas. The off-chain read latency of the same call was negligible throughout (median below 4 ms). The registry is thus inexpensive, and its 50-entry cap is a property of the reference T-REX `TrustedIssuersRegistry` rather than a fundamental limit of the design; it can be relaxed in two independent ways. First, the bound is a single constant in this (non-token) registry contract: because resolution grows only linearly at $\approx 2,268$ gas per issuer, even several hundred accredited issuers would still resolve well within a block, so raising the constant trades a modest, bounded gas increase for a larger set. Second, and more scalably, the registry entries need not be individual institutions. An accreditation *authority* (a ministry or national agency, itself registered as a trusted issuer) can accredit institutions by issuing each an accreditation claim, so that the top-level trusted set stays small regardless of how many institutions participate; a verifier then follows one extra hop, checking that the issuing institution holds a valid accreditation claim from a trusted authority. The present implementation registers institutions directly, which suits a single consortium or region; the authority-based structure is the natural path to national or cross-institutional deployment, and realising this delegated accreditation end-to-end is future work.

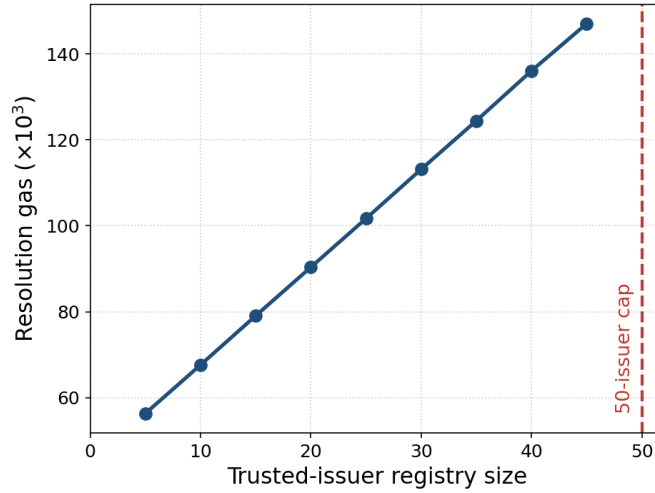


Fig. 5. Issuer-resolution gas as a function of trusted-issuer registry size: linear ($\approx 2,268$ gas/issuer), bounded by the registry's 50-issuer limit.

Claims per identity. We grew a single identity from 5 to 30 certificate claims (each necessarily issued by a distinct claim issuer, since a claim is keyed by `keccak256(issuer, topic)`) and measured both the gas of adding a further claim and the latency of the validation read (Figure 6). Claim addition is effectively constant at 290,756 gas (varying by fewer than 40 gas across all sizes), confirming that issuance cost is independent of how many claims an identity already holds. (This absolute number is lower than the 355,409 headline cold cost of Table 4 because the sweep uses a minimal per-claim payload and a distinct issuer per claim; the invariance, not the absolute value, is the result of interest here.) The validation read, in contrast, grows roughly linearly, from a median of 5.1 ms at 5 claims to 37.9 ms at 30 (about 1.3 ms per claim; 95th percentile up to 49 ms). This growth is not on-chain gas (validation is read-only) but client round-trips: retrieving the claims of a topic issues one call to enumerate the claim identifiers followed by one call per claim. It is therefore network-dependent, and the reported numbers, obtained on an in-process node, are a lower bound; the cost can be flattened by batching the per-claim reads through a multicall, an optimisation we leave as future work. In practice a certificate comprises only a handful of claims, so validation latency remains well within interactive limits.

5.5 Latency and Throughput (RQ4)

Table 7 reports end-to-end latency for issuance and validation, measured on the Sepolia public testnet over $R = 30$ trials. For issuance we measure the interval between transaction submission and first confirmation; for validation we measure the round-trip time of the read-only call that retrieves an identity's certificate claims for a topic. Because confirmation times on public networks are heavy-tailed, we report the median and the 95th percentile.

Issuance latency has a median of 11.9 s, closely tracking Sepolia's ≈ 12 s block interval: the cost is dominated by waiting for block inclusion rather than by execution. Two ≈ 25 s outliers (24.7 and 25.2 s) correspond to missed blocks, and over 30 trials they lift the 95th percentile to 24.7 s. Validation latency has a median of 0.10 s (p95 0.16 s); being a read-only call served by an RPC node, it is independent of block production and roughly two orders of magnitude faster

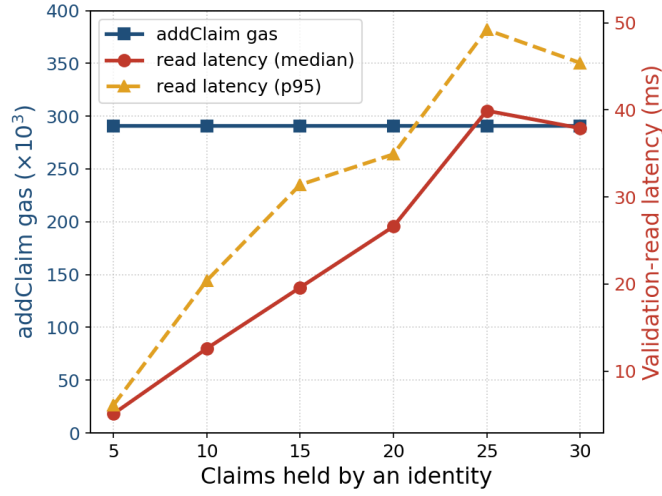


Fig. 6. Claim-addition gas (constant) and validation-read latency (linear in the number of claims held by an identity).

Table 7. End-to-end latency on the Sepolia public testnet ($R = 30$). Issuance is measured from submission to first confirmation; validation is a read-only call and is not subject to block confirmation.

Operation	Median (s)	p95 (s)
Issue certificate	11.91	24.68
Validate certificate	0.10	0.16

Table 8. Measured performance of the four-validator Besu QBFT consortium as the block interval is swept ($N = 8$ concurrent issuers, 1,200 issuances, up to 400 in flight; latency over 20 unloaded sequential issuances). All runs completed with zero failed issuances.

Interval (s)	Latency, med. (s)	Throughput (cert/s)	Gas/block
1	1.06	45.6	71–93%
2	1.94	63.1	71%
5	5.01	41.8	82%

than issuance. The gas consumed on Sepolia matched the deterministic local-network numbers of Table 4 (a cold first claim of $\approx 347k$ gas and warm updates of $\approx 115k$), confirming that gas is chain-independent.

Sustained throughput on a permissioned consortium. Public-network latency is dominated by block times outside the operator’s control; the realistic deployment target is instead the permissioned consortium of Section 5.1, whose block interval is a tunable parameter. We drive it with a load generator that models N accredited institutions issuing concurrently: each of $N = 8$ trusted ClaimIssuers signs from its own account and writes to its own subject identity, and the driver distributes submissions across all four validators’ RPC endpoints so that every rotating QBFT proposer has transactions in its local mempool. We stream 1,200 certificate issuances with up to 400 in flight and record the sustained rate of mined issuances and the per-block gas utilisation; separately, we measure unloaded end-to-end issuance latency over 20 sequential issuances. Because QBFT provides immediate finality, an included transaction is final, so this latency is the user-visible confirmation time. Table 8 reports both as the block interval is swept.

Two effects are visible. First, unloaded issuance latency tracks the block interval almost exactly, a single block to finality, so the interval directly tunes responsiveness. Second, sustained throughput is *not* maximised by the shortest interval: it peaks at ≈ 63 certificates per second at a 2 s interval and falls to ≈ 46 at 1 s, where the modest two-vCPU validators and the cross-region round-trips cannot reliably complete a QBFT round every second under load and the protocol round-changes, raising the effective interval. At 5 s the chain is stable but leaves block time unused. Across all intervals the driver ran to completion with no failed issuances and the mined blocks filled to 71–82% of the 30 M-gas limit (up to ≈ 200 issuances per block), showing that issuance throughput is bounded by the on-chain addClaim state write ($\approx 122k$ gas warm) rather than by consensus or networking. As a reference point, the same harness streaming plain value transfers (21k gas each) on the 2 s chain sustained 114 transactions per second at only 18% gas utilisation: the $\approx 6\times$ heavier T-REX claim write, not the chain, sets the certificate-issuance ceiling. Even the lowest measured rate corresponds to over 3.5 million issuances per day, comfortably above the demand of a national diploma system (which awards on the order of 10^5 degrees per year, well under one issuance per second on average), so the consortium sustains the workload with substantial headroom.

5.6 Security Analysis (RQ5)

We analyse the security of the registry against a threat model covering the principal assets (certificates, issuer keys, and student identities) and the trust assumptions of the design. Table 9 summarises the identified threats, the mechanism that mitigates each, whether the mitigation is exercised by a negative test, and the residual risk. The negative tests are implemented as an executable suite: each attempts a disallowed action (issuance by an unaccredited issuer, on-chain issuance without an authorised identity key, a forged claim signature, use of a revoked claim, issuance by a de-accredited issuer, a duplicate-salt registration, and exceeding the trusted-issuer cap) and asserts that it fails. In particular, one test writes a validly signed certificate claim through a *direct contract call* from a non-accredited issuer, bypassing the application layer entirely, and confirms that the on-chain verification view (isCertifiedBy) rejects it because the issuer is absent from the trusted-issuer registry; companion tests confirm that de-accrediting the issuer (removeTrustedIssuer) or removing the claim (removeClaim) likewise flips the on-chain verdict to invalid. This directly addresses the concern that accreditation might be enforced only in client code: the check is an on-chain invariant of verification, not a client-side policy that a caller could skip. The one threat not covered by a negative test, on-chain privacy leakage, is a cryptographic property (the dictionary-attack resistance of a hash of low-entropy data) rather than an access-control check, and is assessed analytically instead.

Revocation is effective at verification. Because isCertifiedBy reads the *current* registry and identity state, a revoked certificate fails later verification with no additional bookkeeping. Two revocation actions are exercised by the negative-test suite, and each flips the on-chain verdict to invalid: removing the claim (removeClaim) makes it absent, so no matching claim is found; and de-accrediting the issuer (removeTrustedIssuer) drops it from the trusted set, so its claims no longer count, revoking, in a single operation, every certificate that issuer signed. Moreover, for an institution onboarded as a full ClaimIssuer, isCertifiedBy also honours *per-claim signature-based revocation*: revokeClaimBySignature records the claim’s signature on the issuer, and because isCertifiedBy delegates authenticity to the issuer’s isClaimValid, which rejects revoked signatures, the certificate thereafter fails verification even though it remains stored on the subject’s identity and its issuer stays accredited; a dedicated test confirms this. This issuer-side revocation also closes the re-add residual, since re-adding the same signed claim still fails validation.

Table 9. Threat model, mitigations, and residual risk. “Tested” indicates whether a negative test exercises the mitigation; the remaining case (privacy leakage) is a cryptographic property assessed analytically rather than by a revert test.

Threat	Mitigation	Tested	Residual risk
Unauthorised issuance	onlyClaimKey at write; issuer accreditation and isClaimValid enforced on-chain at verification (isCertifiedBy)	Yes	Registry owner is trusted; owner-key compromise defeats accreditation
Issuer-key compromise	ERC-734 key rotation (removeKey); removeTrustedIssuer de-accredits	Yes	Claims signed before detection stay valid until the key or issuer is removed
Forged / replayed claim	ECDSA claim signed over (identity, topic, data); isClaimValid	Yes	No nonce or expiry; a valid signature can be re-added by a claim-key holder
Revocation bypass	removeClaim / removeTrustedIssuer; isCertifiedBy reads current registry and claim state	Yes	A verifier reading stale state may accept a revoked claim
Registration front-running	Owner-gated createIdentity; salt uniqueness (“salt already taken”)	Yes	Owner manages salts; owner-key leak allows identity squatting
Registry-bloat DoS	Trusted-issuer set capped at 50; only accredited issuers write claims	Yes	A trusted issuer can still bloat an identity, raising read latency
Privacy leakage on-chain	AES-256 on sensitive fields; only the SHA-256 digest stored on-chain	No	Hash of low-entropy data is dictionary-attackable; needs salting

The remaining residuals are narrow. A verifier that reads *stale* state rather than current state may accept an already-revoked claim; this is inherent to any cached read and is avoided by querying current state, as `isCertifiedBy` does. And the lightweight identity-proxy onboarding of Section 5.2 uses the base `Identity.isClaimValid`, which has no revoked-signature set, so on that path revocation relies on `removeClaim` and `removeTrustedIssuer` rather than per-claim signature revocation; recovering the latter there, for instance through a nonce or an explicit expiry epoch in the signed claim tuple, remains the one revocation direction we leave as future work. The two onboarding paths thus present a deliberate choice: cheaper onboarding without per-claim signature revocation, or the full `ClaimIssuer` with it.

In addition to the threat-based analysis, we subjected the compiled contract set to static analysis using Slither.⁴ Over the full compilation (133 contracts) the large majority of findings are informational or optimization notes, and the high- and medium-severity findings arise almost entirely within the reused `OnchainID` and `T-REX` contracts (for example reentrancy and unchecked-return-value findings in the token-transfer, delivery, and modular-compliance machinery), which are externally audited implementations that our certificate registry does not exercise (it uses only the identity, claim, and trusted-issuer components, and no token transfers). Beyond that base, the contracts we introduce are three two-function helpers: the certificate verifier (`CertificateVerifier`), a file-storage reference contract, and the hash-anchoring baseline used only for the comparison of Section 5.3. On the certificate verifier, Slither reports three benign issues for a stateless read-only view: an unused return value (the claim tuple is destructured to the four fields the check consumes, namely topic, issuer, signature, and data, intentionally ignoring scheme and uri), two external-call-in-a-loop notes (intrinsic to iterating the trusted-issuer set, which is capped at 50 and mirrors the audited `isVerified` pattern), and a compiler-version pragma shared with the rest of the codebase; the contract holds no state

⁴<https://github.com/crytic/slither>

and performs no transfers. We therefore consider the added attack surface negligible, while noting that reliance on the upstream contracts inherits their residual findings and that this analysis is not a formal verification.

Two longer-term privacy considerations fall outside the revert-testable threat model. First, the encrypted sensitive fields are anchored immutably, so their confidentiality rests on the continued hardness of AES-256; a deployment with a long retention horizon should account for eventual cryptographic degradation (advances in cryptanalysis or large-scale quantum computing) and, accordingly, keep ciphertext off-chain wherever possible, anchoring only integrity references. Second, under the GDPR the encrypted personal data is best regarded as *pseudonymised* rather than anonymous while a decryption key exists, placing it within scope of the right to erasure (the GDPR “right to be forgotten”); an immutable public ledger cannot honour an erasure request directly, whereas a permissioned consortium deployment, whose participants can collectively prune, re-key, or destroy the keys for stored state, offers a more defensible legal posture, reinforcing our conclusion that a consortium network is the realistic target. Our design mitigates but does not eliminate this exposure, since only integrity references and encrypted fields, never plaintext personal data, need be placed on-chain.

5.7 Threats to Validity

Gas measurements are deterministic and network-independent, but depend on the compiler version and optimizer configuration reported in Section 5.1; different settings yield different absolute values while preserving the relative comparisons. Monetary cost depends on gas price and exchange rate and is therefore reported as a range rather than a point estimate. Latency is network-dependent and was measured on a public testnet (Sepolia); absolute values will differ on other deployments, though the qualitative separation between validation (read-only) and issuance (write, one block confirmation) holds. Sustained throughput and issuance latency were measured on a four-validator Besu QBFT consortium built from modest two-vCPU virtual machines across two cloud regions; absolute rates would rise with more capable hardware, additional validators or issuers, or a co-located deployment, but the qualitative findings (latency tracking the block interval, and issuance being bounded by the addClaim state write rather than by consensus) are hardware-independent. The load was generated from a single client machine, which the multi-endpoint submission path mitigates but does not entirely remove as a bottleneck. Finally, the security analysis is limited to the modelled threats and to the coverage of the static-analysis tools, and does not constitute a formal verification of the contracts.

6 Conclusions

The counterfeiting of academic credentials and the absence of a reliable, widely accepted system for issuing and verifying them remain significant problems in the education sector. Existing blockchain-based registries help establish authenticity but typically anchor certificate hashes without binding them to a verifiable identity, without an explicit on-chain mechanism to accredit issuing institutions, and without support for correcting or revoking credentials once issued.

This paper investigated whether an infrastructure designed for regulated financial instruments can be repurposed to close these gaps. We presented the design of a registry for identity-bound academic credentials that composes OnchainID self-sovereign identities (ERC-734/ERC-735) with the T-REX suite (ERC-3643), in which the trusted-issuer registry serves as an on-chain issuer-accreditation whitelist and each certificate is a signed, updatable claim bound to a student’s identity. Claim writes are gated on-chain by per-identity key authorisation, while issuer accreditation is enforced on-chain *at verification* (a certificate counts only if it is signed by a currently accredited issuer, decided by a single read-only call), while sensitive data is kept off-chain and only an integrity-preserving reference is stored on-chain,

so that third parties can verify a certificate without a wallet and without access to personal data. We made explicit the tension between a transferable-security-token standard and non-transferable credentials, identifying which of its guarantees carry over. A reference implementation covering the full certificate life cycle demonstrates the feasibility of the approach, and our evaluation characterises its cost, scalability, latency, throughput, and security, including the overhead relative to a hash-anchoring baseline; on a four-validator Hyperledger Besu QBFT consortium spanning two cloud regions the design sustains tens of certificate issuances per second with single-block finality, bounded by the on-chain claim write rather than by consensus.

Several directions remain for future work. Beyond the four-validator consortium profiled here, the registry could be deployed and measured on a larger, production-grade consortium and on a public Layer-2 to complement the present evaluation; issuer-side signature-based revocation is already enforced at verification for institutions onboarded as full ClaimIssuers, and extending it to the lightweight identity-proxy onboarding, which trades that capability for a measured sixfold gas reduction (815,083 versus 4,881,061 gas), for instance through a per-claim nonce or an explicit expiry epoch in the signed claim tuple, would make replay-resistant revocation uniform across both onboarding paths; the privacy model could be strengthened with selective-disclosure or zero-knowledge proofs of individual fields; and integration with recognised credentialing frameworks, such as W3C verifiable credentials [38] and the eIDAS 2.0 European Digital Identity Wallet [8], would improve interoperability and support real-world adoption by institutions.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Blockcerts. 2026. Blockcerts: The Open Standard for Blockchain Credentials. <https://www.blockcerts.org/guide/>. Accessed 12 July 2026.
- [2] Vitalik Buterin. 2014. Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum White Paper. <https://ethereum.org/en/whitepaper/>. Accessed 12 July 2026.
- [3] Weilin Chan, Keke Gai, Jing Yu, and Liehuang Zhu. 2025. Blockchain-Assisted Self-Sovereign Identities on Education: A Survey. *Blockchains* 3, 1 (2025), 3. doi:10.3390/blockchains3010003
- [4] Tim Daubenschütz and Anders. 2022. ERC-5192: Minimal Soulbound NFTs. Ethereum Improvement Proposals, no. 5192. Final. <https://eips.ethereum.org/EIPS/eip-5192>.
- [5] Bonnie Eisenman. 2017. *Learning React Native: Building Native Mobile Apps with JavaScript* (2nd ed.). O'Reilly Media, Sebastopol, CA, USA.
- [6] European Commission. 2024. Blockchain Strategy. <https://digital-strategy.ec.europa.eu/en/policies/blockchain-strategy>. (accessed on 12 July 2026).
- [7] European Commission. 2024. European Blockchain Services Infrastructure (EBSI). <https://digital-strategy.ec.europa.eu/en/policies/european-blockchain-services-infrastructure>. (accessed on 12 July 2026).
- [8] European Parliament and Council of the European Union. 2024. Regulation (EU) 2024/1183 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework. Official Journal of the European Union, L series, 30 April 2024. (accessed on 12 July 2026).
- [9] Expo. 2026. Expo Documentation. <https://docs.expo.dev/>. Accessed 12 July 2026.
- [10] Expo. 2026. Expo SecureStore. <https://docs.expo.dev/versions/latest/sdk/securestore/>. Accessed 12 July 2026.
- [11] Gilles Grolleau, Tarik Lakhal, and Naoufel Mzoughi. 2008. An Introduction to the Economics of Fake Degrees. *Journal of Economic Issues* 42 (09 2008), 673–693. doi:10.1080/00213624.2008.11507173
- [12] Sérgio Guerreiro, João F Ferreira, Tiago Fonseca, and Miguel Correia. 2022. Integrating an academic management system with blockchain: A case study. *Blockchain: Research and Applications* 3, 4 (2022), 100099. doi:10.1016/j.bcr.2022.100099
- [13] Jacques Hallak and Muriel Poisson. 2007. *Corrupt schools, corrupt universities: What can be done?* International Institute for Educational Planning (IIEP-UNESCO), Paris, France.
- [14] Ross Harmes and Dustin Diaz. 2008. The Factory Pattern. In *Pro JavaScript Design Patterns*. Apress, Berkeley, CA, USA, 93–108. doi:10.1007/978-1-4302-0496-1_7
- [15] Ross Harmes and Dustin Diaz. 2008. The Proxy Pattern. In *Pro JavaScript Design Patterns*. Apress, Berkeley, CA, USA, 197–214. doi:10.1007/978-1-4302-0496-1_13
- [16] Hyperledger Foundation. 2019. Hyperledger Besu. Available At: <https://www.hyperledger.org/projects/besu>. (accessed on 21 August 2026).

- [17] Tarek Kanan, Ahamd Turki Obaidat, and Majduleen Al-Lahham. 2019. SmartCert BlockChain Imperative for Educational Certificates. In *2019 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT)*. 629–633. doi:10.1109/JEEIT.2019.8717505
- [18] Konstantinos Karasavvas. 2018. Revoking Records in an Immutable Ledger: A Platform for Issuing and Revoking Official Documents on Public Blockchains. In *2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*. 105–111. doi:10.1109/CVCBT.2018.00019
- [19] Michael Kuperberg. 2020. Blockchain-Based Identity Management: A Survey From the Enterprise and Ecosystem Perspective. *IEEE Transactions on Engineering Management* 67, 4 (2020), 1008–1027. doi:10.1109/TEM.2019.2926471
- [20] Meta Platforms. 2026. React Native. Available at: <https://reactnative.dev/>. (accessed on 12 July 2026).
- [21] Abdulhafour Mohammad and Sergio Vargas. 2022. Barriers Affecting Higher Education Institutions' Adoption of Blockchain Technology: A Qualitative Study. *Informatics* 9, 3 (2022). doi:10.3390/informatics9030064
- [22] Abdulhafour Mohammad and Sergio Vargas. 2022. Challenges of Using Blockchain in the Education Sector: A Literature Review. *Applied Sciences* 12, 13 (2022). doi:10.3390/app12136380
- [23] Richard Moore. 2021. Ethers. Available at: <https://ethers.org/>. (accessed on 12 July 2026).
- [24] Richard Moore. 2023. Ethers Documentation. Available at: <https://docs.ethers.org/v6/>. (accessed on 12 July 2026).
- [25] Mozilla. 2024. JavaScript. Available at: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/>. (accessed on 12 July 2026).
- [26] Alexander Mühle, Andreas Grüner, Tatiana Gayvoronskaya, and Christoph Meinel. 2018. A Survey on Essential Components of a Self-Sovereign Identity. *Computer Science Review* 30 (2018), 80–86. doi:10.1016/j.cosrev.2018.10.002
- [27] Satoshi Nakamoto. 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. <https://bitcoin.org/bitcoin.pdf>. (2008). Accessed 12 July 2026.
- [28] Nomic Foundation. 2026. Hardhat. <https://hardhat.org/>. Accessed 12 July 2026.
- [29] Jack E. Olson. 2009. Chapter 5 - Origins of a Database Archiving Application. In *Database Archiving*, Jack E. Olson (Ed.). Morgan Kaufmann, Boston, 71–84. doi:10.1016/B978-0-12-374720-4.00005-4
- [30] OnchainID. 2024. OnchainID. Available at: <https://docs.onchainid.com/>. (accessed on 12 July 2026).
- [31] OnchainID. 2024. ONCHAINID: Proposal and Architecture. Available at: <https://docs.tokeny.com/docs/general-concept>. (accessed on 12 July 2026).
- [32] Jae Park. 2021. Promises and challenges of Blockchain in education. *Smart Learning Environments* 8, 1 (2021), 33. doi:10.1186/s40561-021-00179-2
- [33] P Paul, PS Aithal, Ricardo Saavedra, and Surajit Ghosh. 2021. Blockchain technology and its types—a short review. *International Journal of Applied Science and Engineering (IJASE)* 9, 2 (2021), 189–200.
- [34] Avni Rustemi, Fislak Dalipi, Vladimir Atanasovski, and Aleksandar Risteski. 2023. A Systematic Literature Review on Blockchain-Based Systems for Academic Certificate Verification. *IEEE Access* 11 (2023), 64679–64696. doi:10.1109/ACCESS.2023.3289598
- [35] Omar S Saleh, Osman Ghazali, and Muhammad Ehsan Rana. 2020. Blockchain based framework for educational certificates verification. *Journal of critical reviews* (2020). doi:10.1109/ICESC51422.2021.9532968
- [36] Tokeny Solutions. 2023. T-Rex Purpose and Architecture. Available at: <https://docs.tokeny.com/docs/purpose-architecture-1>. (accessed on 12 July 2026).
- [37] Manu Sporny, Dave Longley, and David Chadwick. 2022. *Verifiable Credentials Data Model v1.1*. W3C Recommendation. World Wide Web Consortium (W3C). (accessed on 12 July 2026).
- [38] Manu Sporny, Ted Thibodeau Jr., Ivan Herman, Gabe Cohen, and Michael B. Jones. 2025. *Verifiable Credentials Data Model v2.0*. W3C Recommendation. World Wide Web Consortium (W3C). Published 15 May 2025. (accessed on 12 July 2026).
- [39] Gautami Tripathi, Mohd Abdul Ahad, and Gabriella Casalino. 2023. A comprehensive review of blockchain technology: Underlying principles and historical background with future challenges. *Decision Analytics Journal* 9 (2023), 100344. doi:10.1016/j.dajour.2023.100344
- [40] Muhamed Turkanović, Marko Hölbl, Kristjan Košič, Marjan Heričko, and Aida Kamišalić. 2018. EduCTX: A Blockchain-Based Higher Education Credit Platform. *IEEE Access* 6 (2018), 5112–5127. doi:10.1109/ACCESS.2018.2789929
- [41] Fabian Vogelsteller. 2017. ERC: Claim Holder. Available at: <https://github.com/ethereum/EIPs/issues/735>. (accessed on 12 July 2026).
- [42] Fabian Vogelsteller. 2017. ERC: Key Manager. Available at: <https://github.com/ethereum/EIPs/issues/734>. (accessed on 12 July 2026).
- [43] Paul Voigt and Axel von dem Bussche. 2017. *The EU General Data Protection Regulation (GDPR): A Practical Guide* (1st ed.). Springer Publishing Company, Incorporated.
- [44] WalletConnect. 2024. WalletConnect. Available at: <https://docs.walletconnect.com/>. (accessed on 12 July 2026).
- [45] E. Glen Weyl, Puja Ohlhaber, and Vitalik Buterin. 2022. Decentralized Society: Finding Web3's Soul. Available at SSRN: <https://ssrn.com/abstract=4105763>. (accessed on 12 July 2026).
- [46] Erna Widyasari, Ahmad Muhibbin, Iyad Abdallah Al-Shreifeen, et al. 2024. Blockchain in Education: Potentials and Challenges in Academic Records and Certifications. *International Journal of Teaching and Learning* 2, 5 (2024), 1252–1266.
- [47] Gavin Wood. 2014. Ethereum: A Secure Decentralised Generalised Transaction Ledger. Ethereum Yellow Paper. <https://ethereum.github.io/yellowpaper/paper.pdf>. Accessed 12 July 2026.
- [48] Yang Xiao, Ning Zhang, Wenjing Lou, and Y Thomas Hou. 2020. A survey of distributed consensus protocols for blockchain networks. *IEEE Communications Surveys & Tutorials* 22, 2 (2020), 1432–1465. doi:10.1109/COMST.2020.2969706
- [49] Zibin Zheng, Shaoan Xie, Hongning Dai, Xiangping Chen, and Huaimin Wang. 2017. An overview of blockchain technology: Architecture, consensus, and future trends. In *2017 IEEE international congress on big data (BigData congress)*. IEEE, 557–564. doi:10.1109/BigDataCongress.2017.85